

LEGB

 ZADANIE

Zaprezentować i przestudiować przykład demonstrujący zakresy L – local, E – enclosed, G – global, B – builtin

```
import builtins

x = "GLOBAL x"  # G - global

def outer():
    x = "OUTER x"  # L dla outer, E (enclosing) dla funkcji inner

    def inner_local():
        # L - tu definiujemy własne lokalne x
        x = "INNER x"
        print("L (inner_local):", x)

    def inner_enclosing():
        # brak lokalnego x → Python szuka:
        # L (brak) → E (x z outer) → G → B
        # global x - można wskazać przed użyciem
        # nonlocal x - albo tak można wskazać przed użyciem
        print("E (inner_enclosing):", x)
        x += " inner_enclosing"

    inner_local()
    inner_enclosing()
    # lokalne w outer, enclosing dla inner_*
    print("outer (swoje lokalne x):", x)

print("G (przed outer):", x)
outer()
print("G (po outer):", x)
```

```
def demo_builtins():
    # Zasłaniamy nazwę len lokalnie:
    len = "LOCAL len"
    print("L (shadowed len):", len)
    # A tu jawnie wołamy funkcję wbudowaną:
    print("B (builtins.len):", builtins.len([1, 2, 3]))

demo_builtins()
```

Zmienna `__builtins__`

```
print(__builtins__)
```

w globalnym zasięgu:

- w normalnym skrypcie zwykle będzie to **moduł builtins**,
- w niektórych środowiskach (np. interpreter interaktywny, IDE, Jupyter) może to być **słownik** z zawartością builtins.

```
# przypadek 1 - __builtins__ jest modułem
__builtins__.len([1, 2, 3])
```

```
# przypadek 2 - __builtins__ jest dict-em
__builtins__["len"]([1, 2, 3])
```

MRO

 ZADANIE

Zadanie ma na celu sprawdzenie jak wygląda tworzenie obiektów dla typów z wielokrotnym dziedziczeniem, jakie funkcje `__new__` oraz `__init__` są lub nie są wywoływane. Wychodzimy od dwóch klas bazowych (identycznych, różnią się tylko nazwą), `class Baza(object)` oraz `class A(object)`. Posiadają one napisane `__new__`, `__init__`, `__str__` oraz funkcję `id()`. Warto przestudiować kilka różnych wariantów klas potomnych (B, C, D...) oraz tworzenia odpowiednich obiektów. Proponowane scenariusze są zapisane poniżej, klasy potomne powinny mieć zawartość zbliżoną, w celach studyjnych, do klas bazowych.

```
class Baza(object):
    def __new__(cls, *args):
        print("> Baza __new__", *args)
        nowy_obiekt = object.__new__(cls)
        print("<- Baza __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("> Baza __init__", x)
        super().__init__()
        print("<- Baza __init__")
        self.x = x
        print("<- Baza __init__")
    def __str__(self):
        return "{self.x}".format(self=self)
    def id(self):
        print("-Baza-")
```

```
class A(object):
    def __new__(cls, *args):
        print("> A __new__", *args)
        nowy_obiekt = super().__new__(cls)
        print("<- A __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("> A __init__", x)
        super().__init__(x)
        print("<- A __init__")
        self.x = x
        print("<- A __init__")
    def __str__(self):
        return "{self.x}".format(self=self)
    def id(self):
        print("-A-")
```

```
class B(Baza):
    pass

class C(B):
    pass

class D(A, C, B, Baza):
    # tu nie definiować __new__
    pass
```

```
### SCENARIUSZ 1:
# print(B.mro())
# b = B(123)
# print("-----")
# b.id()
# print("-----")
# print(b)
```

```
### SCENARIUSZ 2:
# print(C.mro())
# c = C(456)
# print("-----")
# c.id()
# print("-----")
# print(c)
```

```
### SCENARIUSZ 3:
# print(D.mro())
# d = D(789)
# print("-----")
# d.id()
# print("-----")
# print(d)

### SCENARIUSZ 4:
# tak jak 3, tylko zobaczyć, co się dzieje podczas rzutowania:
# A(d),id() albo B(d),id() itp.
```

warto zwrócić uwagę na niuanse
(miejsca zaznaczone strzałkami), wywołania:

- `object.__new__` lub `super().__new__`
- przekazanie bądź nie argumentu `x`